Ride Sensor Capture

A multimodal mobile platform for two-wheeler telemetry, event labeling, edge inference, predictive road-hazard alerts, and ML-ready dataset generation

Abstract. Ride Sensor Capture is a Flutter-based research application that synchronizes vehicle dynamics, rider physiology, GPS, and edge-camera detections into a single time-aligned dataset. It connects ESP32-S3 wearable IMUs and Polar Verity Sense sensors over BLE, records trips and event windows, extracts physics and machine-learning features, classifies live riding events, learns geo-tagged hazards, warns the rider before repeat encounters, gathers post-event feedback, and exports reproducible JSON/CSV research artifacts. This report documents the complete implemented feature set, system architecture, algorithms, data schema, user interfaces, verification evidence, and current scientific limitations.

01 System scope & contribution
02 Architecture & feature map
03 Sensor acquisition & protocols
04 Processing & edge inference
05 Event labeling & ride analytics
06 Predictive alert learning loop
07 Persistence & dataset export
08 Verification & limitations

Evaluated build

App 1.0.1 (version code 2)
Commit 0e174bd
Android API 24–36
Report date 21 Oct 2026

Flutter / Dart | BLE + GPS + PPG + IMU | Drift / SQLite WAL | Groq-assisted feedback

Development attribution: rashiyaom

1 · SYSTEM SCOPE AND RESEARCH CONTRIBUTION

From heterogeneous sensors to an auditable riding dataset

The platform targets a practical gap in two-wheeler research: synchronized acquisition of road excitation, vehicle motion, rider response, context, and human ground truth without a laboratory data logger. A journey is treated as the primary experimental unit. During an active journey, BLE packets, GPS fixes, event labels, camera detections, derived parameters, warnings, and rider feedback share UTC timestamps and trip/event identifiers. This permits both live safety behavior and later supervised-learning analysis.

9	50–52 Hz	1.5 s
SQLITE RESEARCH TABLES	NOMINAL IMU INGESTION	ROLLING ML WINDOW

224	35,570	6
AUTOMATED CHECKS PASSED	PRODUCTION DART LOC	EVIDENCE SCREENSHOTS

End-to-end functional path

ESP32 / Polar / GPS / camera › Parse, normalize & time-align › Filter, label & infer › SQLite WAL knowledge base › Alerts, feedback & export

Implemented feature inventory

Subsystem	Implemented capability	Research value
Acquisition	Concurrent BLE discovery/connection, ESP32 3/6-axis frames, Polar HR/RR and PMD acceleration, GPS route fixes, HTTP camera events, demo sources.	Multimodal evidence under a common trip timeline.
Reliability	Reconnect backoff, packet-gap health, sequence continuity, write buffering, WAL, session alerts, battery status, permission banner.	Captures provenance and quality threats during field trials.
Ground truth	Tap, quick-trigger, continuous voice command, and watch-button event control; automatic 60 s cap; event-bound sensor association.	Low-interruption annotation during riding.
Signal analysis	Unit normalization, calibration/noise estimation, zero-phase filters, jerk, lean, g-force, cross-sensor lag/correlation, HR-response gate, FFT/statistical windows.	Produces interpretable physics and tabular ML features.
Live inference	Five anomaly classes plus normal riding, evaluated every 200 ms on a 75-sample buffer; confidence and explanatory features displayed live.	Immediate event screening at the edge.
Safety loop	Persistent road-event registry, heading/radius filtering, time-to-event warning, TTS, speed advice, warning log, contextual feedback, Groq/offline scoring.	Links detection, intervention, and outcome assessment.
Dataset delivery	Raw time series, events, trips, pre-alerts, effectiveness, windowed features, nested JSON, Markdown report, and baseline Python trainer.	Shortens the path from ride collection to model experiment.

Software architecture

Presentation & control

Dashboard · Devices · Events · Export · Journey log · event details · warning banner · feedback sheet

Reactive application layer

Riverpod providers · trip/event controllers · BLE-to-DB bridge · watch synchronization · export controller

Domain & analytics

Parsers · parameter engine · filters · cross-validation · feature extraction · alert engine · feedback analyzer

Interfaces

BLE GATT · Geolocator · speech-to-text · TTS · local HTTP server · share sheet

Persistence

Drift-generated typed schema · SQLite WAL · repository abstraction · migration version 10

External endpoints

ESP32 watches · Polar Verity · camera firmware · OpenStreetMap tiles · optional Groq API

Audit basis. This report was derived from all 133 Dart source/test files, Android configuration, generated Drift schema, export code, and executed verification artifacts at commit `0e174bd` .

Field-ready sensing across vehicle, rider, position, and vision

ESP32-S3 watch / IMU nodes

The BLE parser recognizes service `FFE0` and characteristic `FFE1`. It accepts ASCII CSV plus 12, 16, 19–23, 24, 28, and 32-byte binary frames. The V2 frame carries timestamp, sequence, role, six float channels, and battery data; role byte 1 maps to fork and 2 to footboard. Legacy accelerometer values within a plausible ±8 g range are normalized to m/s². Non-finite or malformed frames are dropped.

Polar Verity Sense

Standard Heart Rate service `180D/2A37` supports 8/16-bit BPM and RR/PPI conversion from 1/1024 s units to milliseconds. Polar PMD data `fb005c82…` yields signed 16-bit XYZ acceleration converted from mg using 0.00981 m/s² per mg. The app requests and visualizes heart rate, PPI, and motion streams.

Connection lifecycle and watch companion

- 15 s BLE discovery, explicit connect/disconnect, multi-device state map, mount-location assignment, and simulated 50 Hz sources for controlled demonstrations.
- 10 s connection timeout; at most six automatic reconnect attempts with exponential delays starting at 2 s and capped at 20 s.
- Health inspection every 1.5 s; packet gaps above 2.5 s are degraded and above 15 s are poor. Sequence numbers and per-device packet rates expose loss.
- Bidirectional watch packets remain within the default 20-byte ATT payload and synchronize trip state, active event, speed, distance, HR, GPS lock, time, elapsed time, watch face, and brightness.

GPS and edge-camera channel

Navigation-quality GPS fixes provide latitude, longitude, altitude, speed, heading, and accuracy. The app derives speed variance, acceleration, overspeed state, route distance, and route history. An embedded Shelf HTTP server listens on port `8765`: `POST /camera-event` validates device, class, confidence, timestamp, and bounding box; `GET /health` reports liveness. Detections are linked to an active event or to a completed event within ±10 s.

> **Permissions and platform contract.** Android declares BLE scan/connect/advertise, fine/coarse/background location, Internet/network, wake lock, foreground service, connected-device service, and location service. TTS service discovery is declared for Android 11+.

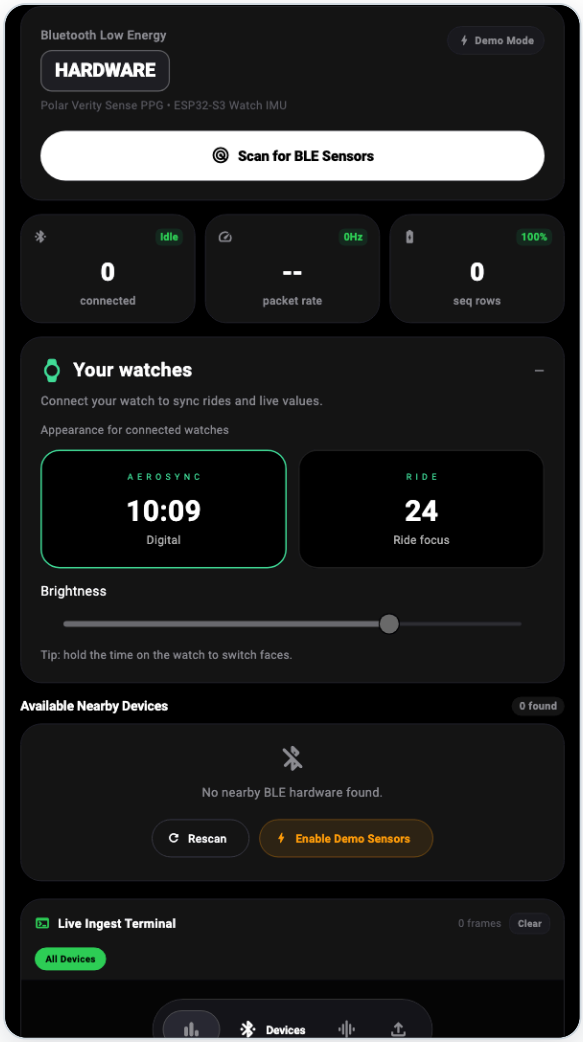


Figure 1. Implemented device hub showing BLE scanning, hardware/demo modes, connection telemetry, watch appearance/brightness controls, nearby devices, and the raw ingest terminal. Captured from the Flutter widget tree.

Acquisition records retained

Modality	Primary stored fields
IMU	XYZ acceleration, XYZ gyroscope, device/mount, sequence, raw payload
Physiology	BPM, PPI, device source, timestamps
GPS	Coordinates, speed, heading, altitude, accuracy
Vision	Class, confidence, camera/receive time, bounding box, event link

Interpretable physics and low-latency anomaly screening

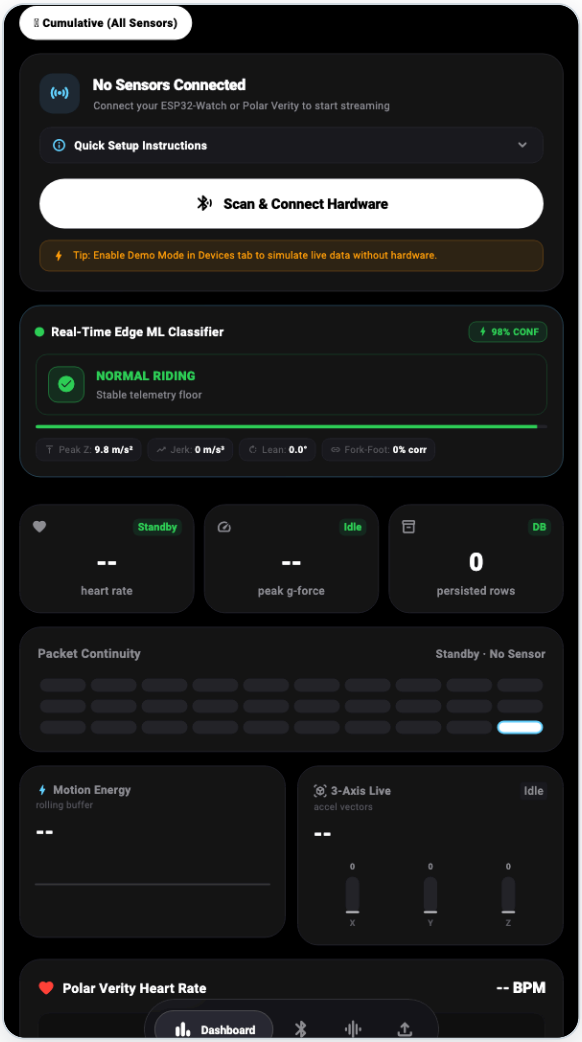


Figure 2. Dashboard evidence: sensor status, setup guidance, real-time classifier with confidence and explanatory features, packet continuity, energy/axis views, heart rate, and floating navigation.

Live dashboard functions

The dashboard combines an analog/digital GPS speedometer, live route map, journey start/stop, sample-rate selector, multi-axis waveforms, 3-axis bars, g-force/jerk/lean metrics, packet-continuity grid, database row counts, battery/session-health banners, Polar heart-rate view, active pre-alert banner, and access to the journey log. UI telemetry is throttled to protect rendering while ingestion continues at sensor rate.

Conditioning and derived physics

Stage	Implementation
Normalization	Acceleration in m/s²; angular helpers enforce rad/s↔deg/s consistency; gravity constant 9.80665 m/s².
Calibration	Per-trip, per-mount idle calibration estimates engine-noise frequency/amplitude and validity.
Filtering	Zero-phase second-order Butterworth: 3 Hz low-pass for maneuver/lean channels and 0.4 Hz high-pass for bump transients; optional adaptive notch only when engine noise is >5 Hz and below Nyquist; reflection padding reduces edge artifacts.
Impact	3-point magnitude smoothing, peak g, peak-to-peak change, acceleration delta, short-window RMS, and unattenuated 3-axis jerk.
Turning	Complementary roll estimate (gyro integration + accelerometer roll, α=0.98), peak lateral acceleration, duration, and GPS heading change.
Validation	Fork/footboard peak agreement: bump window ±300 ms with ≥1.25 g on both sensors; turn/brake window ±1000 ms with ≥2 m/s² lateral/longitudinal response.
Physiology	60 s pre-event median HR baseline; 5 s response window; ≥8 BPM rise marks physiological confirmation.

On-device inference rules

A 75-sample (1.5 s at 50 Hz) fork buffer, optional synchronized footboard buffer, and 200 ms evaluation cadence produce a six-state output. Features include vertical maximum, peak-to-peak range, standard deviation, peak jerk, roll, longitudinal acceleration, yaw rate, and fork/footboard correlation.

```
severe bump: (zPeak > 18 OR zPTP > 16) AND jerk > 120
mild bump: (zPeak > 13.5 OR zPTP > 7.5) AND jerk > 50
sharp turn: |roll| > 12° OR |yaw| > 22°/s
hard braking: ax < −3.0 m/s² · rapid acceleration: ax > +2.5 m/s²
```

Confidence is a bounded monotonic function of threshold exceedance and the interface exposes both confidence and a natural-language explanation. The offline feature exporter separately forms 1.5 s windows with 0.5 s stride, statistical/FFT/vector features, dual-watch cross-correlation, labels, and deterministic 70/15/15 train/validation/test partitions.

> **Scientific interpretation.** The current “edge ML” runtime is an explainable rules ensemble, not a learned neural model. Its thresholds are unit-tested and useful as a baseline/label-quality screen; external validation and calibration are still required before claiming clinical or safety efficacy.

Low-interruption annotation linked to complete trip context

Journey lifecycle

Starting a journey creates a trip row, opens GPS tracking, activates high-frequency persistence, synchronizes the watch, clears session counters, starts health monitoring, and arms predictive alerts. During the ride, route points and telemetry update distance, duration, average/peak speed, event count, sensor-row count, physiology, and behavior aggregates. Ending a journey flushes pending packets, stops tracking, computes final statistics, closes the pre-alert session, and makes the trip available in the Journey Log Book.

Four complementary event-input paths

1. **Tap-to-mark:** the rider selects bump, turn, or hard brake; a prominent STOP action completes the window.
2. **Quick triggers:** one-tap BUMP, TURN, HARD BRAKE, and custom labels reduce interaction time.
3. **Voice commands:** continuous speech maps configured phrases to start/stop operations and exposes recognized text/listener state.
4. **Watch companion:** watch control packets can start/stop labels; serialized writes and phone/watch state echoes prevent overlapping events.

Each active window receives an event ID before incoming sensor rows are written. On stop, ingestion detaches from the event, buffered rows are flushed, start/end GPS is captured, physics parameters are computed, the trip event count is updated, and the state briefly reports “saved.” A 60 s timer prevents indefinitely open windows; serialized controller operations protect rapid consecutive labels.

Event parameterization and classification

Label	Parameters and decision thresholds
Bump	Peak acceleration/g-force, Δa, peak-to-peak, duration, peak jerk, baseline noise. Moderate at ≥1.4 g or ≥22 m/s³; severe at ≥2.2 g or ≥45 m/s³, with calibration-adaptive g limits.
Turn	Peak lateral acceleration, complementary-filter lean, duration, direction/heading. Medium at ≥2.5 m/s² or ≥14°; sharp at ≥5 m/s² or ≥28°.
Speed/brake	Peak/average speed where present, longitudinal acceleration/deceleration, jerk, duration, and braking classification.
Evidence flags	Cross-confirmed state, fork–footboard lag, HR spike, HR delta, GPS speed/heading change, calibration validity, engine-noise frequency.

Post-capture inspection

The event history shows type, time, classification, peak metric, and feedback badge. Detail views expose the full computed-parameter JSON, GPS coordinates with external map launch, and device/mount-selectable XYZ traces downsampled to at most 150 points for display. Trips support route maps, BLE quality summaries, per-journey export, notes, deletion, and pruning of unlabeled telemetry after export.

> **Traceability.** Every labeled row retains both `tripId` and `eventId`; labels therefore remain joinable to raw samples, GPS, camera evidence, pre-alerts, feedback, and trip-level covariates.

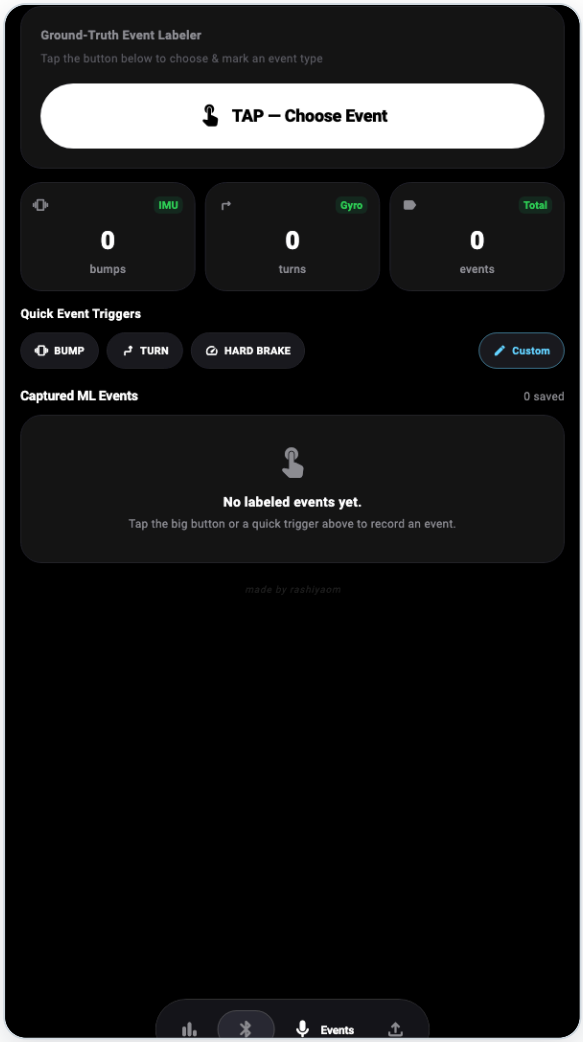


Figure 3. Event Labeler evidence with tap-to-record, three quick event classes, custom labels, counters, history state, and voice-oriented navigation.

Closing the loop from hazard observation to intervention outcome



Figure 4. Warning evidence: hazard identity, distance, time-to-event, and recommended speed surfaced while GPS tracking is active.

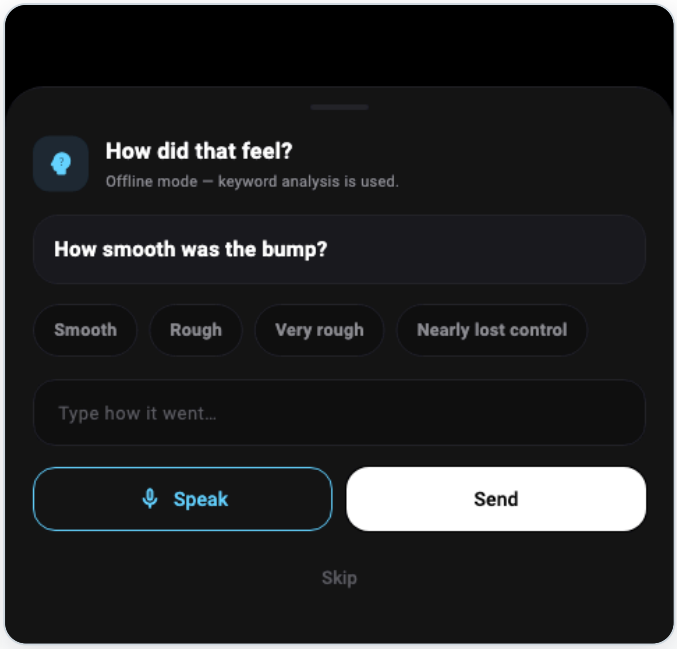


Figure 5. Feedback evidence: contextual prompt, quick replies, free text, speech capture, send/skip controls, and offline-analysis status.

Hazard knowledge base and alert decision

Confirmed live-inference anomalies and completed labeled events create `road_events` rows with type, coordinate, severity, confidence, source, trip, speed, timestamp, counts, and notes. At 1 Hz GPS cadence the service queries a bounding box, applies exact haversine range, rejects speeds below 8 or above 160 km/h, ignores hazards outside 800 m, removes hazards behind the rider when heading is known (±70° forward cone), and ranks remaining candidates by arrival time.

```
time-to-event = distance / (speed ÷ 3.6)
speak when 2 s ≤ TTE ≤ 20 s
urgency: imminent ≤4 s · warning ≤9 s · advisory >9 s
```

Target pass speed interpolates between 55 km/h for a mild event and 25 km/h for a severe one, constrained by a 1.2 s reaction budget and 3.5 m/s² comfortable deceleration. The message builder produces concise speech; the dashboard mirrors it visually. Issued warnings are attributed to the active trip, persisted before later evaluation, deduplicated per hazard, and protected by a two-minute re-alert cooldown.

Feedback capture and evaluation

After an event is saved, the app links the most recent warning within ten minutes and asks a type-specific question: bump/pothole smoothness, turn difficulty, or generic navigation difficulty. Riders may use one-tap replies, typed text, one-shot dictation, or skip. The raw response is stored before analysis.

The evaluator requests structured JSON from Groq (`openai/gpt-oss-120b` by default) and records warning effectiveness [0–1], timing adequacy, difficulty, summary, improvement suggestion, confidence, model provenance, and timestamps. Network/auth/rate/format failure falls back to a deterministic keyword-and-telemetry heuristic so the capture workflow remains functional without cloud access.

Compiled effectiveness evidence

The report builder joins alerts and feedback, then aggregates total/evaluated alerts, mean effectiveness, mean lead time, mean warning distance, timing distribution, per-event-type counts/effectiveness, and actionable insights. Results appear in the Export screen, JSON top-level `pre_alert_effectiveness`, CSV, and a Markdown report.

> **Privacy/security condition.** Feedback sent to Groq leaves the device; the key used by a mobile client is recoverable from the compiled APK. Conference deployment should use explicit consent and a restricted, proxy-managed, or short-lived credential. No claim of autonomous collision avoidance is made.

Typed, joinable records from raw packets to intervention assessment

Drift / SQLite data model (schema v10)

SQLite runs in write-ahead logging mode. The repository buffers incoming packets and commits at 250 ms intervals or 25 rows, reducing per-packet transaction overhead while retaining an explicit flush at lifecycle boundaries. UTC timestamps, sequence numbers, mount roles, device identity, trip IDs, and event IDs preserve alignment and provenance.

Table	Purpose and key content
`sensor_readings`	Device/type/mount, sequence, UTC, modality, trip/event, HR/PPI, 6-axis IMU, GPS speed, raw payload.
`trips`	Rider/wrist, temporal bounds, route, distance/speed, event/row counts, behavior counts, HR, night %, driver score, notes.
`location_readings`	Trip-linked UTC coordinate, altitude, speed, heading, accuracy.
`event_records`	Label window, trigger, GPS endpoints, status, computed JSON, peak/class, cross/HR/jerk/GPS evidence.
`camera_detections`	Device, class/confidence, camera/receive timestamps, bounding box, linked event.
`trip_calibrations`	Trip/mount engine-noise frequency, amplitude, time, validity.
`road_events`	Geo-tagged hazard type, severity/confidence/source, detection context, alert/feedback counts.
`pre_alerts`	Hazard/trip/event, kinematics, TTE, advised speed, urgency, exact speech, acknowledgement.
`event_feedbacks`	Prompt/response/source plus LLM/heuristic effectiveness, timing, difficulty, summary and recommendation.

Export modes and research artifacts

The researcher chooses events-only or full raw stream, JSON/CSV/both, time range, and optional cross-confirmed-only filtering. Export version 2.1 produces:

- `sensor_timeseries_export.csv` with nearest/forward-filled GPS; `event_records_export.csv`; `trip_summary_export.csv`.
- `features_windowed_session.csv` containing statistical, FFT, orientation, vector, correlation, physiology, label ID, evidence flags, and split membership.
- `pre_alerts_export.csv`, `alert_effectiveness_export.csv`, and `alert_effectiveness_report.md`.
- Nested JSON preserving trips, locations, sensor arrays, events, camera detections, pre-alerts, feedback, schema metadata, and report aggregates.
- A generated `train_baseline.py` script for a scikit-learn RandomForest baseline, enabling a first reproducible classifier experiment directly from the export.

> **Research-readiness feature.** The export screen retains generated archives, summarizes rows/labels/sensors, shares multiple files through the operating system, and exposes alert-effectiveness statistics alongside ML datasets.

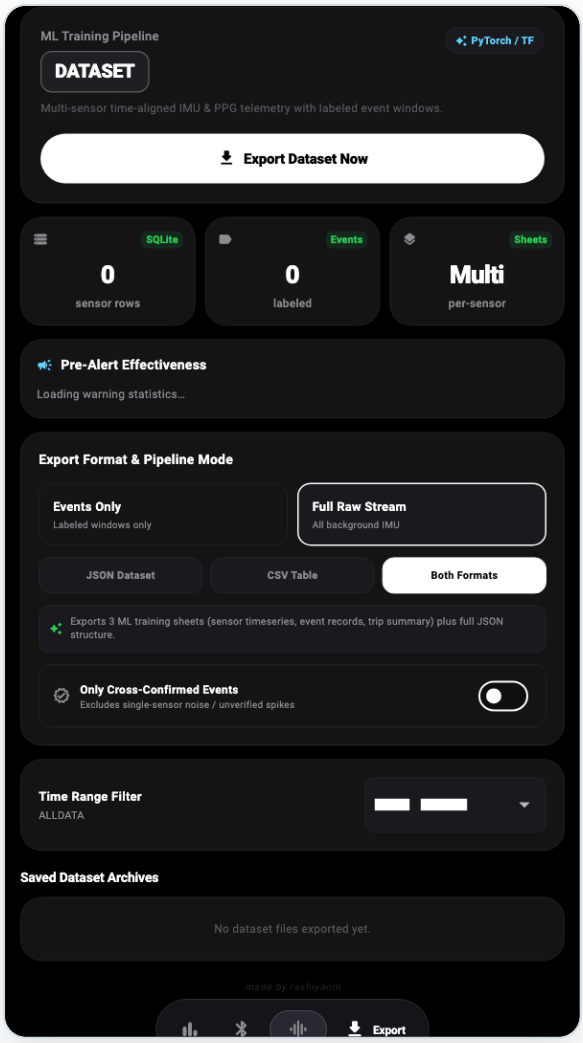


Figure 6. Export evidence showing format/mode selection, cross-confirmation filtering, time range, pre-alert effectiveness, dataset summaries, and saved archives.

Evidence of implementation quality and boundaries of interpretation

0	224 / 224	65.7 MB
FLUTTER ANALYZER ISSUES	EXECUTED TESTS PASSED	RELEASE APK BUILT

Automated verification coverage

Verification area	Representative assertions exercised
Protocol and ingestion	ESP32 frame variants/units/corrupt data; BLE demo streams and reliability; watch protocol and bidirectional lifecycle; Polar parsing; GPS association; camera persistence/linking.
Storage and lifecycle	Batch repository behavior; trip create/start/end aggregates; event serialization under rapid commands; database joins; deletion/pruning paths.
Signal and analytics	Filter behavior, g/rad unit consistency, jerk, lean fusion, FFT features, window labeling/splits, cross-correlation, idle calibration, HR gate, live anomaly rules.
Alert and feedback	Geodesy, time-to-event/window/urgency/speed advice, heading filtering, deduplication, audit/event linking, warning UI, contextual feedback, Groq request/JSON/error paths, offline heuristics, report aggregates.
Export and presentation	CSV/JSON schemas, camera bounding boxes, pre-alert/effectiveness files, baseline trainer, speedometer/map/widgets, complete feature validation, and app navigation smoke test.

✓ **Static analysis:** `flutter analyze` completed with “No issues found.” ✓ **Tests:** `flutter test` completed with “All tests passed” at 224 checks. ✓ **Cloud path:** a live sample was analyzed successfully by the configured Groq model and returned structured, LLM-marked feedback. ✓ **Android artifact:** universal APK (arm64-v8a, armeabi-v7a, x86_64), package `com.example.ride_sensor_capture`, min API 24, target API 36; APK Signature Scheme v2 verified.

Limitations that should accompany conference claims

- **No field-performance dataset is bundled.** The report proves implemented behavior and software tests, not sensitivity/specificity on representative roads or riders.
- **Inference is threshold-based.** “ML” refers to feature production and a rules baseline at runtime; learned-model deployment remains future work.
- **Timestamp source.** Several BLE parsers use phone receipt time rather than reconstructing device clock, so radio/OS latency can affect fine-grained synchronization.
- **GPS/heading uncertainty.** Haversine distance and current heading do not model road topology; parallel roads and noisy heading can create false candidate matches.
- **Hazard consolidation.** Time/class deduplication exists, but repeated passes can create spatially nearby rows without cluster fusion.
- **Cloud data governance.** Groq analysis requires consent, network access, credential protection, retention documentation, and a non-client-secret design for deployment.
- **Human factors.** Spoken warnings and feedback prompts require controlled evaluation for distraction, timing, comprehension, accessibility, and false-alarm burden.
- **Distribution signing.** The evaluated APK uses the project’s debug signing key; production distribution requires a protected release key and stable application ID.

Recommended conference evaluation protocol

1. Calibrate every mount and document device placement, firmware, sampling frequency, phone model, and clock policy.
2. Collect stratified routes across surface types, speeds, traffic, day/night, riders, and repeated hazard passes; annotate with independent video and two raters.
3. Report packet loss, GPS accuracy, sensor synchronization error, label agreement, class balance, event-level precision/recall/F1, onset timing error, and leave-one-rider/route-out validation.
4. For alerts, report warning lead-time distribution, correct-road association, repeated-alert burden, speed reduction, subjective difficulty, and paired passes with/without warning.

Conclusion. Ride Sensor Capture is a technically complete research acquisition and prototyping platform: it unifies heterogeneous sensing, interpretable ride-event analytics, proactive hazard warnings, rider feedback, and reproducible exports in one mobile workflow. Its strongest contribution is traceability across the full loop—raw evidence → label/inference → warning → rider outcome. The next scientific step is controlled field validation, spatial hazard clustering, device-clock synchronization, and replacement/comparison of the runtime rules with trained models.

Evidence method. Figures 1–6 were rendered from the repository’s actual Flutter widgets at a fixed phone/tablet viewport using production theme components and real Material/Roboto fonts. They are not artist-generated interface mockups. Source review and verification were performed on 21 October 2026.